

Práctica 2 - Capa de Aplicación

HTTP

Introducción

1. ¿Cuál es la función de la capa de aplicación?

Cito el libro, capítulo 1:

“La capa de aplicación es donde residen las aplicaciones de red y sus protocolos. La capa de aplicación de Internet incluye muchos protocolos, tales como el protocolo HTTP (que permite la solicitud y transferencia de documentos web), SMTP (que permite la transferencia de mensajes de correo electrónico) y FTP (que permite la transferencia de archivos entre dos sistemas terminales). Veremos que determinadas funciones de red, como la traducción de nombres legibles que utilizamos las personas para los sistemas terminales de Internet (por ejemplo, www.ietf.org) en direcciones de red de 32 bits se realiza también con la ayuda de un protocolo específico de la capa de aplicación, en concreto el Sistema de nombres de dominio (DNS, Domain name system). (...) Un protocolo de capa de aplicación está distribuido a lo largo de varios sistemas terminales, estando la aplicación en un sistema terminal que utiliza el protocolo para intercambiar paquetes de información con la aplicación de otro sistema terminal. A este paquete de información de la capa de aplicación lo denominaremos mensaje”.

Entonces, ¿función de la capa de aplicación?

Su función es definir y proveer los protocolos necesarios para que las aplicaciones de dos computadoras distintas se puedan comunicar entre sí a través de la red. No contiene las aplicaciones en sí (como el navegador web o cliente de correo electrónico), sino que proporciona los servicios necesarios para que se comuniquen a través de la red. Ergo la **capa de aplicación** es una capa de **abstracción** que especifica los **protocolos de comunicación** compartidos y funciona como **interfaz** para las USN aplicaciones abstrayendo al programador de la complejidad de la transferencia de datos subyacente

2. Si dos procesos deben comunicarse:

a. ¿Cómo podrían hacerlo si están en diferentes máquinas?

Cito al libro, capítulo 2.1.2:

“Los procesos de dos sistemas terminales diferentes se comunican entre ellos intercambiando mensajes a través de la red de computadoras. Un proceso emisor crea y envía mensajes a la red; un proceso receptor recibe estos mensajes y posiblemente responde devolviendo mensajes.”

b. Y si están en la misma máquina, ¿qué alternativas existen?

Cito al libro, capítulo 2.1.2:

“En la jerga de sistemas operativos, realmente no son los programas sino los procesos los que se comunican (...) Cuando los procesos se ejecutan en el mismo sistema terminal, pueden comunicarse entre sí mediante la comunicación entre procesos aplicando las reglas gobernadas por el sistema operativo del sistema terminal”

3. Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la "vida cotidiana" y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

Cito al libro, capítulo 2.1.2:

“Una aplicación de red consta de parejas de procesos que se envían mensajes entre sí a través de una red. En el contexto de una sesión de comunicación entre una pareja de procesos, el proceso que inicia la comunicación (es decir, el que inicialmente se pone en contacto con el otro proceso al principio de la sesión) se etiqueta como el cliente. El proceso que espera a ser contactado para comenzar la sesión es el servidor”.

- **Servidor:** Es un **host que está siempre activo** ("always-on host") y cuya función es **atender las solicitudes de muchos otros hosts**, que son los clientes. Estos servidores suelen ser equipos más potentes que almacenan y distribuyen recursos como páginas web. Una característica clave es que el servidor tiene una **dirección IP fija y conocida**, lo que permite a los clientes contactarlo fácilmente.
- **Cliente:** Son los hosts que **inician la comunicación** al ponerse en contacto con el servidor. Típicamente, los clientes son computadoras de escritorio y notebooks, smartphones, etc.. En esta arquitectura, los clientes **no se comunican directamente entre sí**. Cuando un servidor recibe una solicitud de un cliente, responde enviándole el objeto o servicio solicitado.

Ejemplo de un sistema Cliente/Servidor en la "vida cotidiana"

Un buen ejemplo sería una **biblioteca física**.

- **Servidor:** Sería el **sistema de catalogación y gestión de la biblioteca**, incluyendo al personal y los estantes con los libros. Este "sistema" está siempre activo durante el horario de atención y contiene todos los recursos (libros, información) para los usuarios.
- **Cliente:** Sería una **persona (usuario)** que va a la biblioteca con la intención de buscar un libro. El cliente inicia la interacción (pregunta por un libro, busca en el catálogo) y el "servidor" de la biblioteca le proporciona el servicio (le indica dónde está el libro, se lo presta). La persona no se comunica directamente con otros usuarios para conseguir el libro, sino que interactúa con el sistema central de la biblioteca.

Ejemplo de un sistema informático que siga el modelo Cliente/Servidor

Uno de los ejemplos más clásicos y omnipresentes es la **Web**.

- Cuando se utiliza un **navegador web** (como Chrome o Firefox) en su computadora o teléfono, el navegador actúa como el **proceso cliente**.
- Al escribir una dirección URL o hacer clic en un enlace, el navegador envía **mensajes de solicitud HTTP** a un **servidor web** (el proceso servidor).
- El servidor web, que está siempre activo y tiene una dirección IP fija, recibe estas solicitudes y responde con **mensajes de respuesta HTTP** que contienen los objetos solicitados, como páginas HTML, imágenes, etc..

¿Conoce algún otro modelo de comunicación?

Sí, otro modelo de comunicación de aplicaciones fundamental que contrasta con el modelo Cliente/Servidor es:

- **Modelo Peer-to-Peer (P2P - par-a-par):**
 - A diferencia de la arquitectura cliente-servidor, el modelo P2P **depende mínimamente (o en absoluto) de una infraestructura de servidores siempre activos**.
 - En su lugar, una serie de hosts conectados de forma intermitente, llamados **pares (peers)**, se comunican directamente entre sí.
 - Un proceso en un sistema P2P puede actuar tanto como **cliente** (descargando archivos) como **servidor** (cargando archivos).
 - Ejemplos de aplicaciones que utilizan una arquitectura P2P incluyen la **compartición de archivos** (como Torrent), o **videollamadas** (como Skype).

4. Describa la funcionalidad de la entidad genérica "Agente de usuario" o "User agent".

Un agente de usuario (o user agent) es la aplicación de software que actúa en representación de una persona para interactuar con los protocolos y servicios de una red. Su funcionalidad varía según el contexto de la aplicación que se esté utilizando.

En el entorno del correo electrónico, los agentes de usuario permiten a las personas leer, responder, reenviar, guardar y redactar mensajes. Un ejemplo puede ser Gmail. Cuando un usuario termina de redactar un correo, el agente de usuario se encarga de enviar el mensaje al servidor de correo

En el entorno de la Web (utilizando el protocolo HTTP), el agente de usuario especifica el tipo y versión del navegador que está realizando la solicitud al servidor, como por ejemplo un navegador Firefox (Mozilla/5.0). Esta información se envía a través de un header User-agent: dentro del mensaje de solicitud HTTP.

Pero aquí no nos están preguntando ni por correo electrónico ni por HTTP, la pregunta es de la ENTIDAD GENÉRICA, así que la respuesta son los 3 renglones de arriba, no hay que confundir los conceptos, que se prestan a confusión.

5. ¿Qué son y en qué se diferencian HTML y HTTP?

Capítulo 2.2.1 del libro:

HTML es un **estándar para el formato de documentos** utilizado en la Web. Las páginas web (o documentos) suelen estar compuestas por un archivo HTML base que estructura el contenido y puede hacer referencia a múltiples objetos adicionales, como imágenes o secuencias de video.

Por otro lado, **HTTP** (*HyperText Transfer Protocol*) es el **protocolo de la capa de aplicación** que opera en el corazón de la Web. Su función principal es definir la estructura, el formato y la secuencia de los mensajes que se intercambian entre los clientes (como un navegador web) y los servidores web. Básicamente, HTTP dicta las reglas sobre cómo un cliente solicita una página web y cómo el servidor transfiere esa página de vuelta al cliente.

6. HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas. (Ayuda: apartado “Formato de mensaje HTTP”, Kurose).

a. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información? ¿Para qué sirven las cabeceras?

La información que indica si un mensaje HTTP es de requerimiento o de respuesta se encuentra en la **primera línea del mensaje**.

Esta primera se compone de la siguiente manera:

- **Mensaje de requerimiento (Request):** Su primera línea se denomina **línea de solicitud** (*request line*). Está compuesta por tres campos: el **campo del método** (que puede tomar valores como GET, POST, HEAD, PUT y DELETE), el **campo de la URL** que identifica el objeto solicitado, y el **campo de la versión de HTTP**.
- **Mensaje de respuesta (Response):** Su primera línea se denomina **línea de estado inicial** (*status line*). Está compuesta también por tres campos: el **campo de la versión del protocolo**, un **código de estado** (por ejemplo, 200, 301, 404) y un **mensaje de estado** correspondiente (por ejemplo, OK, Moved Permanently, Not Found).

Las cabeceras (o líneas de cabecera) sirven para transmitir metadatos e información adicional necesaria para que tanto el cliente como el servidor configuren y entiendan el contexto de la transacción.

- **En los requerimientos:** Las cabeceras le permiten al cliente brindarle directivas y preferencias al servidor. Por ejemplo, se utiliza Host: para especificar en qué host reside el objeto, Connection: para definir si la conexión TCP debe cerrarse o mantenerse abierta, User-agent: para identificar el tipo de navegador (lo que permite al servidor enviar versiones de la página adaptadas a distintos dispositivos), y Accept-language: para negociar el idioma preferido.
- **En las respuestas:** Las cabeceras le proporcionan al cliente detalles sobre el servidor y el objeto que se está enviando. Ejemplos de esto son Date: (fecha y hora en que se creó la respuesta), Server: (software que generó la respuesta, como Apache), Content-Length: (tamaño del objeto en bytes) y Content-Type: (tipo de contenido, como texto HTML). También se incluye Last-Modified:, que indica cuándo fue la última vez que el objeto se modificó y es un dato crítico para el correcto funcionamiento de la caché web.

b. ¿Cuál es su formato? (Ayuda:

<https://developer.mozilla.org/es/docs/Web/HTTP/Headers>)

El formato de una cabecera HTTP consiste en una línea de texto con la siguiente estructura simple:

nombre : valor

1. Nombre: Es una cadena de caracteres insensible a mayúsculas (case-insensitive).
2. Dos puntos (:): Actúan como separador obligatorio entre el nombre y el valor.
3. Valor: El contenido específico de la cabecera. Aunque el valor puede contener espacios, los espacios en blanco que aparecen justo antes o después del valor se ignoran.

Cada línea de cabecera termina con un carácter de retorno de carro seguido de un salto de línea (CRLF). El bloque completo de cabeceras finaliza con una línea vacía (un CRLF adicional) para indicar el fin de los metadatos y el inicio del cuerpo del mensaje, si lo hay.

c. Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como www.misitio.com para obtener el recurso `/index.html`. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento

```
GET /index.html HTTP/1.1
Host: www.misitio.com
User-Agent: curl/7.74.0
```

7. Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s).

- **-I (o --head)**: Le pide al servidor que envíe **únicamente los encabezados** de la respuesta (el *header*), sin descargar el cuerpo del mensaje (HTML, imagen, etc.). Útil para chequear códigos de estado rápidamente.
- **-H (o --header)**: Sirve para enviar **encabezados personalizados** en el requerimiento.
- **-X (o --request)**: Permite especificar el **método HTTP** que querés usar (GET, POST, PUT, DELETE, etc.). Si no lo ponés, curl suele usar GET por defecto.
- **-s (o --silent)**: Activa el **modo silencioso**. Oculta la barra de progreso y los mensajes de error, dejando la terminal limpia solo con la respuesta del servidor.

8. Ejecute el comando curl sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:

a. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (>) del comando curl a un archivo con extensión html y abrirlo con un navegador.

Se realizó un solo requerimiento HTTP (un método GET por defecto). Al ejecutar el comando sin parámetros adicionales hacia la URL, curl establece una conexión TCP, envía la solicitud y espera la respuesta.

Se recibió el cuerpo del mensaje de respuesta HTTP, que en este caso es el código fuente HTML de la página.

b. ¿Cómo funcionan los atributos href de los tags link e img en html?

El atributo href (Hypertext Reference) en HTML define la ruta o URL de destino para vincular recursos externos, principalmente usado en la etiqueta <a> para enlaces y <link> para archivos CSS o iconos. Funciona indicando al navegador dónde encontrar el recurso, ya sea localmente (ruta relativa) o en internet (URL absoluta).

Diferencia con : La etiqueta no utiliza href. Utiliza el atributo src (source) para especificar la ruta de la imagen a mostrar

c. Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?

No, no alcanza con un único requerimiento.

Cuando se realiza un pedido HTTP (como el curl de recién), el servidor solo envía el documento HTML base. Este archivo es puramente texto pero sí contiene las rutas (en los atributos src o href) de los demás elementos. Hay que realizar luego un requerimiento por cada uno de esos recursos.

Si una página tiene 10 imágenes y 2 archivos CSS, el navegador hace al menos 13 requerimientos en total (1 para el HTML + 12 para los objetos adicionales) para poder renderizar la página completa. Curl trae solo lo que le pedimos explícitamente (en este caso, solo el HTML).

d. ¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos CSS, dos Javascript y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando curl ejecutado previamente.

Para esta página específica, serían necesarios 8 requerimientos HTTP en total para obtenerla de forma completa.

1 para el HTML, y 1 para cada uno de los elementos mencionados en el enunciado (son 7).

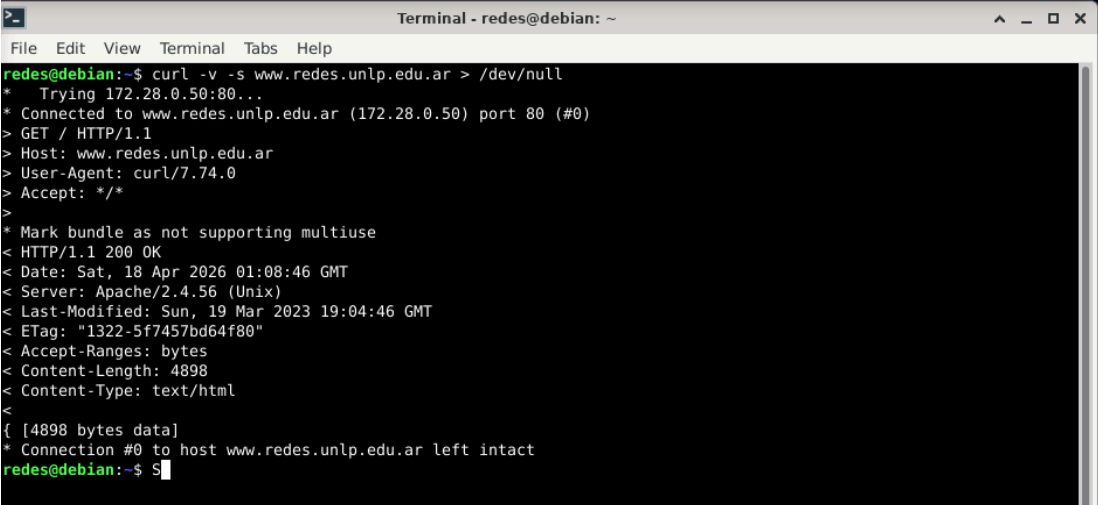
9. Ejecute a continuación los siguientes comandos:

- `curl -v -s www.redes.unlp.edu.ar > /dev/null`
- `curl -I -v -s www.redes.unlp.edu.ar`

10. ¿Qué diferencias nota entre cada uno?

`curl -v -s www.redes.unlp.edu.ar > /dev/null` :

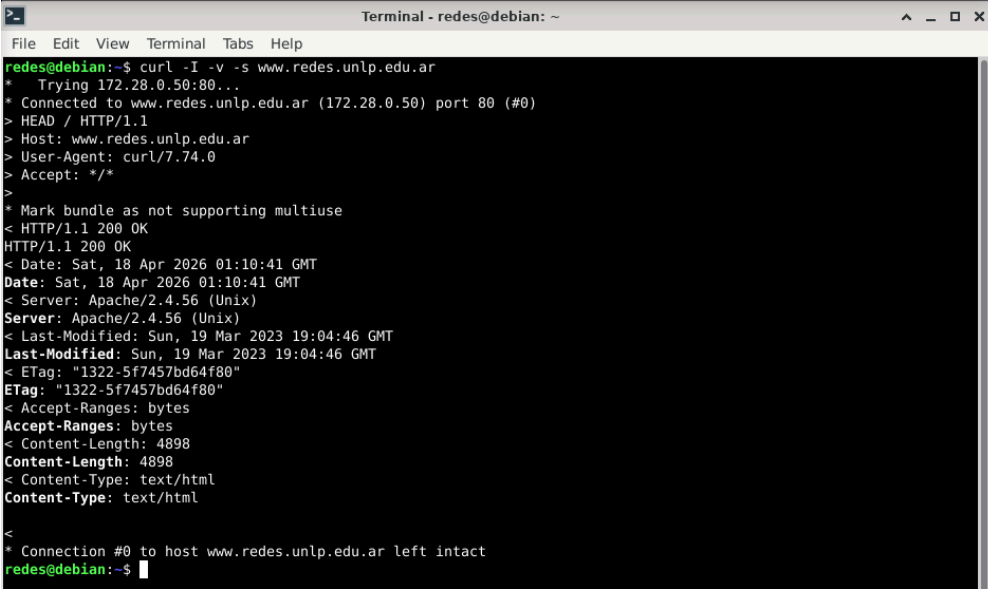
- Utiliza el método **GET**. El GET solicita al servidor el recurso completo (cabeceras + cuerpo/HTML). Sin embargo, al usar la redirección `> /dev/null`, el cuerpo de la respuesta se descarta y no se muestra en pantalla. Solo vemos las cabeceras porque el parámetro `-v` (verbose) las envía a la salida de error estándar (stderr), que no es afectada por la redirección simple.



```
Terminal - redes@debian: ~
File Edit View Terminal Tabs Help
redes@debian:~$ curl -v -s www.redes.unlp.edu.ar > /dev/null
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sat, 18 Apr 2026 01:08:46 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
[ 4898 bytes data]
* Connection #0 to host www.redes.unlp.edu.ar left intact
redes@debian:~$
```

`curl -I -v -s www.redes.unlp.edu.ar` :

- El parámetro `-I` cambia el método de GET a HEAD. En este caso, el cliente le pide al servidor que solo envíe las cabeceras de respuesta, sin el cuerpo del mensaje.

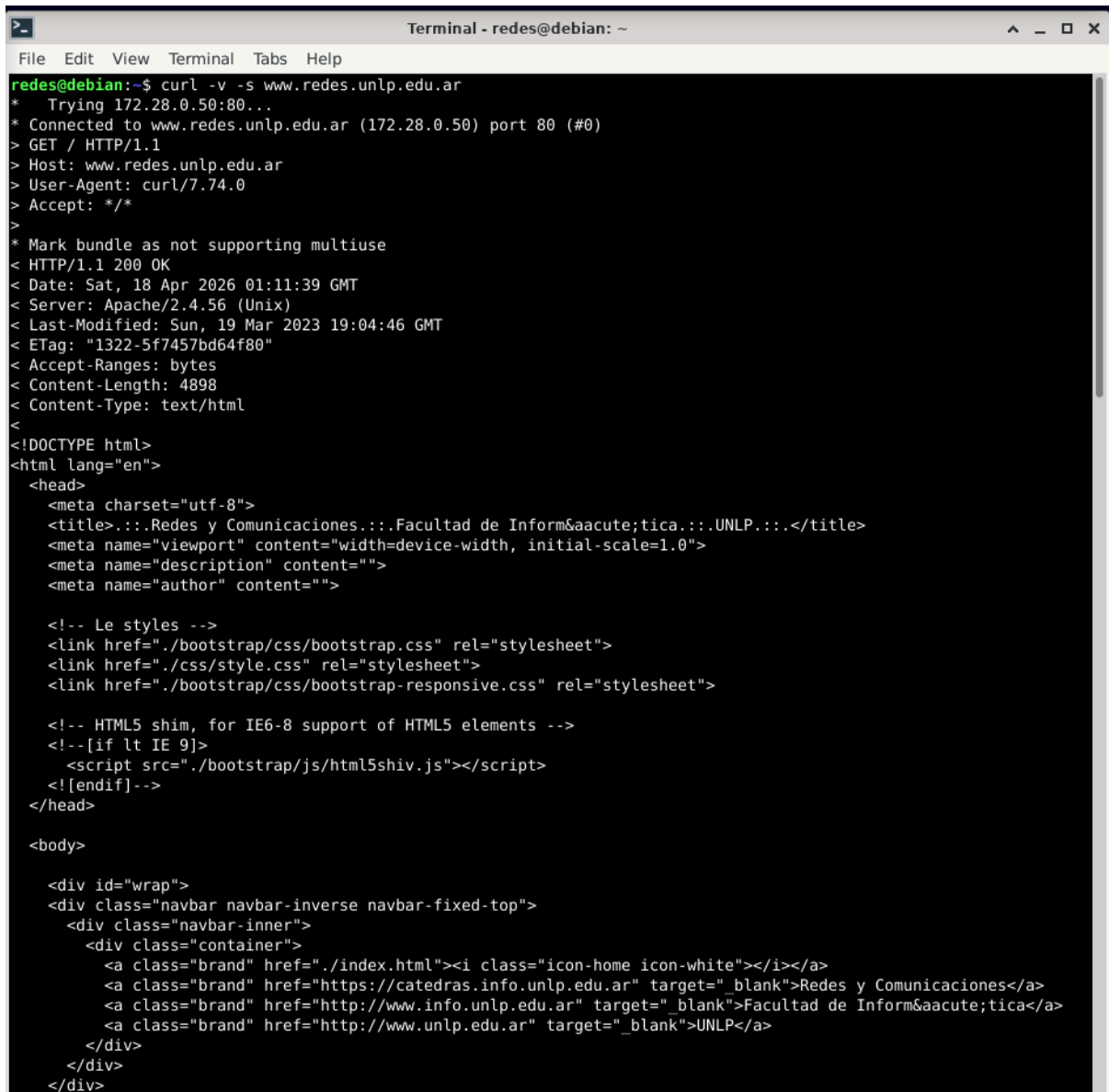


```
Terminal - redes@debian: ~
File Edit View Terminal Tabs Help
redes@debian:~$ curl -I -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> HEAD / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
HTTP/1.1 200 OK
< Date: Sat, 18 Apr 2026 01:10:41 GMT
Date: Sat, 18 Apr 2026 01:10:41 GMT
< Server: Apache/2.4.56 (Unix)
Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
Accept-Ranges: bytes
< Content-Length: 4898
Content-Length: 4898
< Content-Type: text/html
Content-Type: text/html
<
* Connection #0 to host www.redes.unlp.edu.ar left intact
redes@debian:~$
```

b. ¿Qué ocurre si en el primer comando se quita la redirección a /dev/null? ¿Por qué no es necesaria en el segundo comando?

Si sacamos la redirección en el primer comando la terminal muestra todo el HTML de la página web inmediatamente después de las cabeceras.

Al ser un método GET, el servidor envía el archivo completo y, sin la redirección, curl lo imprime directamente en la consola.



```
Terminal - redes@debian: ~
File Edit View Terminal Tabs Help
redes@debian:~$ curl -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sat, 18 Apr 2026 01:11:39 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>:::Redes y Comunicaciones:::Facultad de Inform&aacute;tica:::UNLP:::</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">

    <!-- Le styles -->
    <link href="/bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="/css/style.css" rel="stylesheet">
    <link href="/bootstrap/css/bootstrap-responsive.css" rel="stylesheet">

    <!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
    <!--[if lt IE 9]>
      <script src="/bootstrap/js/html5shiv.js"></script>
    <![endif]-->
  </head>
  <body>

    <div id="wrap">
      <div class="navbar navbar-inverse navbar-fixed-top">
        <div class="navbar-inner">
          <div class="container">
            <a class="brand" href="/index.html"><i class="icon-home icon-white"></i></a>
            <a class="brand" href="https://catedras.info.unlp.edu.ar" target="_blank">Redes y Comunicaciones</a>
            <a class="brand" href="http://www.info.unlp.edu.ar" target="_blank">Facultad de Inform&aacute;tica</a>
            <a class="brand" href="http://www.unlp.edu.ar" target="_blank">UNLP</a>
          </div>
        </div>
      </div>
    </div>
  </body>
</html>
```

¿Por qué no es necesaria en el segundo?: Porque en cURL, [el parametro -I](#) aplica el método HEAD, garantiza que el servidor no enviará el cuerpo del mensaje, solo pide las cabeceras. Al no haber datos de "contenido" (HTML), no hay nada que ocultar o redireccionar a /dev/null; la única información que viaja son las cabeceras que querías consultar. (ver link al manual)

c. ¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?

Puede verse en la captura anterior que el CMD con > marca el requerimiento, y con < marca la rta.

En el requerimiento:

1. Host: www.redes.unlp.edu.ar
2. User-Agent: curl/7.74.0
3. Accept: */*

En la respuesta:

1. Date (Fecha y hora de la respuesta)
2. Server (server del servidor: Apache/2.4.56)
3. Last-Modified (ultima modificación del archivo, sirve para caché)
4. ETag (identificación de versión del recurso)
5. Accept-Ranges (Indica que acepta peticiones parciales en bytes)
6. Content-Length (tamaño del archivo: 4898 bytes)
7. Content-Type (Tipo de contenido: text/html)

10. ¿Qué indica la cabecera Date?

La cabecera Date es un campo estándar en los mensajes HTTP que indica la fecha y hora exacta en la que se originó el mensaje.

Sirve como referencia fundamental para la validación de la **caché** y por estandar, siempre está indicada en horario GMT, para la sincronización temporal entre cliente y servidor.

11. En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?

HTTP/1.0: Conexiones No Persistentes

- En esta versión, por defecto, se abre una conexión TCP para **cada** objeto.
- ¿Cómo sabe el cliente que terminó?: El servidor envía los datos y, al finalizar, cierra la conexión TCP. Cuando el cliente detecta que el socket se cerró, asume que ya recibió todo lo que había para recibir.

HTTP/1.1: Conexiones Persistentes

Acá la conexión se mantiene abierta para mandar varios objetos seguidos (ahorrando tiempo, ya que el handshake lleva bastante tiempo).

Como el "cierre de conexión" ya no sirve de aviso, se usan dos mecanismos:

- **Caso A (Tamaño conocido):** El servidor usa la cabecera Content-Length.
 - Indica el número exacto de bytes del cuerpo. El cliente simplemente va contando los bytes que entran; cuando llega al número indicado, sabe que ese objeto terminó y puede esperar el siguiente.
- **Caso B (Tamaño desconocido/Contenido dinámico):**
 - Se usa la cabecera Transfer-Encoding: chunked.
 - El servidor manda el objeto "de a cachitos". Cada bloque (chunk) empieza con su tamaño en hexadecimal, seguido de los datos.
 - Se indica con un **último** chunk de tamaño 0. Cuando el cliente ve ese cero, sabe que no viene nada más y que el servidor terminó.

12. Investigue los distintos tipos de códigos de retorno de un servidor web y su significado. Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).

Los códigos de estado o de retorno son la forma en que el servidor le cuenta al cliente qué pasó con la solicitud.

Se dividen en cinco categorías principales, identificadas por el primer dígito:

1XX: Informativos

Indican que la petición fue recibida y el proceso continúa. Son poco comunes de ver.

- 100 Continue: El servidor recibió las cabeceras y el cliente debe seguir enviando el cuerpo de la petición.
- 101 Switching Protocols: El servidor acepta cambiar el protocolo (por ejemplo, pasar de HTTP a WebSockets).

2XX: Éxito *(tomá acá tenés)*

Significa que la acción solicitada fue recibida, entendida y aceptada con éxito.

- 200 OK: La respuesta estándar para peticiones exitosas.
- 201 Created: La petición tuvo éxito y se creó un nuevo recurso (común en POST).
- 204 No Content: La petición se completó con éxito pero no hay contenido que devolver.

3XX: Redirección *(no maestro, ese trámite es en el subsuelo)*

El cliente necesita realizar pasos adicionales para completar la solicitud.

- 301 Moved Permanently: El recurso se movió a una nueva URL para siempre.
- 302 Found: El recurso está temporalmente en otra ubicación.
- 304 Not Modified: Indica que el recurso no cambió desde la última vez (para ahorrar ancho de banda usando la caché).

4XX: Errores del Cliente *(me mandaste cualquier cosa)*

Indican que hubo un problema con la petición enviada por el cliente.

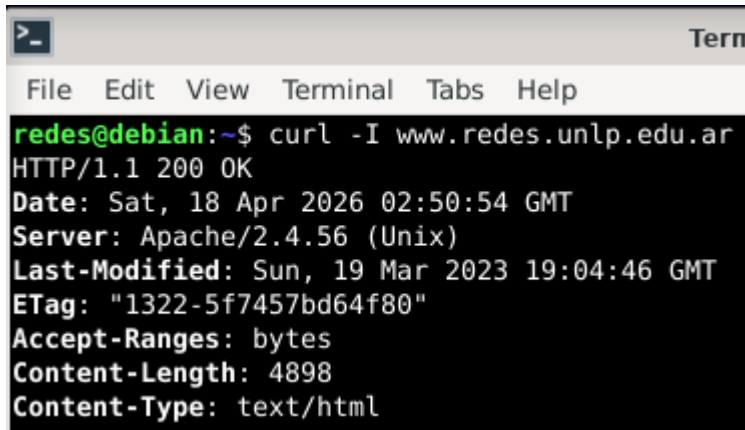
- 400 Bad Request: El servidor no entiende la petición por un error de sintaxis.
- 401 Unauthorized: Falta autenticación o es incorrecta.
- 403 Forbidden: El servidor entiende la petición, pero se niega a autorizarla (no tenés permisos).
- 404 Not Found: El servidor no pudo encontrar el recurso solicitado.

5XX: Errores del Servidor *(no sos vos, soy yo (me rompí))*

El servidor falló al intentar procesar una petición aparentemente válida.

- 500 Internal Server Error: Un error genérico; algo se rompió en el código del servidor.
- 502 Bad Gateway: El servidor, actuando como proxy, recibió una respuesta inválida de otro servidor superior.
- 503 Service Unavailable: El servidor no está listo para manejar la petición (por mantenimiento o sobrecarga).
- 504 Gateway Timeout: El servidor no recibió una respuesta a tiempo de otro servidor.

13. Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:

A screenshot of a terminal window titled 'Terminal'. The prompt is 'redes@debian:~\$'. The command entered is 'curl -I www.redes.unlp.edu.ar'. The output is an HTTP response header: 'HTTP/1.1 200 OK', 'Date: Sat, 18 Apr 2026 02:50:54 GMT', 'Server: Apache/2.4.56 (Unix)', 'Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT', 'ETag: "1322-5f7457bd64f80"', 'Accept-Ranges: bytes', 'Content-Length: 4898', and 'Content-Type: text/html'.

a. ¿Qué información brinda la primera línea de la respuesta?

Indica:

- 1) Versión del protocolo (HTTP/1.1)
- 2) Código de estado 200 (que salió todo bien)
- 3) Mensaje de estado OK

b. ¿Cuántos encabezados muestra la respuesta?

La respuesta muestra 7 encabezados.

- Date (Fecha y hora de la respuesta)
- Server (server del servidor: Apache/2.4.56)
- Last-Modified (ultima modificación del archivo, sirve para caché)
- ETag (identificación de versión del recurso)
- Accept-Ranges (Indica que acepta peticiones parciales en bytes)
- Content-Length (tamaño del archivo: 4898 bytes)
- Content-Type (Tipo de contenido: text/html)

c. ¿Qué servidor web está sirviendo la página?

Servidor Apache/2.4.56

14. Utilizando curl, acceda al sitio

www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados.

```
Terminal - redes@debian: ~
File Edit View Terminal Tabs Help
redes@debian:~$ curl www.redes.unlp.edu.ar/restringido/index.php
<h1>Acceso restringido</h1>

<p>Para acceder al contenido es necesario autenticarse. Para obtener los datos de acceso seguir las instrucciones deta
p>
redes@debian:~$ curl www.redes.unlp.edu.ar/obtener-usuario.php
<p>Para obtener el usuario y la contraseña haga un requerimiento a esta página seteando el encabezado 'Usuario-Redes'
redes@debian:~$ curl www.redes.unlp.edu.ar/obtener-usuario.php -H "Usuario-Redes: obtener"
<p>Bien hecho! Los datos para ingresar son:

    Usuario: redes

    Contraseña: RYC

    Ahora vuelva a acceder a la página inicial con los datos anteriores.

    PISTA: Investigue el uso del encabezado Authorization para el método Basic. El comando base64 puede ser de ayuda!<
redes@debian:~$ echo -n "redes:RYC" | base64
cmVkZXM6UllD
redes@debian:~$ curl -v www.redes.unlp.edu.ar/restringido/index.php -H "Authorization: Basic cmVkZXM6UllD"
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/index.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXM6UllD
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 302 Found
< Date: Sat, 18 Apr 2026 03:32:13 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Location: http://www.redes.unlp.edu.ar/restringido/the-end.php
< Content-Length: 230
< Content-Type: text/html; charset=UTF-8
<
<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACION: la URL de la siguiente página está contenida en esta misma respuesta.</p>

* Connection #0 to host www.redes.unlp.edu.ar left intact

redes@debian:~$ curl -v http://www.redes.unlp.edu.ar/restringido/the-end.php -H "Authorization: Basic cmVkZXM6UllD"
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/the-end.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXM6UllD
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sat, 18 Apr 2026 03:33:40 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Content-Length: 159
< Content-Type: text/html; charset=UTF-8
<
¡Felicitaciones, llegaste al final del ejercicio!

Fecha: 2026-04-18 03:33:40
* Connection #0 to host www.redes.unlp.edu.ar left intact
Verificación: 4325d7704cf7ea00a5ad1dc360b4c9fac5cb4d35a8ab5b5820691a5c5e04c8ffredes@debian:~$ S$
```

15. Utilizando la VM, realice las siguientes pruebas:

a. Ejecute el comando 'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt' y copie la salida completa (incluyendo los dos saltos de línea del final).

```
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*
```

b. Desde la consola ejecute el comando telnet www.redes.unlp.edu.ar 80 y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de hacerlo?

Se ejecutó la petición HTTP.

Mientras que curl es un user-agent (o chrome) que automatizan en proceso, hacerlo con telnet es como hacerlo más manualmente, hay que escribir todo enterito, pero eso:

- Con el comando del enunciado abrimos una conexión TCP contra el puerto 80 del servidor.
- El mensaje Connected to... indica que el Handshake de TCP fue exitoso. Ahí está abierto el pipeline cliente-servidor.
- Ahí le podés mandar el GET con los headers
- El servidor no va a responder hasta que recibe una línea en blanco (por eso nos aclaran que incluyamos el doble enter del final, es la indicación del cliente al servidor de que ya le mandaste todos los headers que le querías mandar)
- Luego el servidor cortó la conexión (porque en la línea del GET indicamos versión 1.0, que tiene conexión NO persistente)

c. Repita el proceso anterior, pero copiando la salida del recurso /extras/prueba-http-1-1.txt. Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando telnet nuevamente.

16. En base a lo obtenido en el ejercicio anterior, responda:

a. ¿Qué está haciendo al ejecutar el comando telnet?

Al ejecutar telnet `www.redes.unlp.edu.ar 80`, estamos estableciendo una conexión TCP (capa de transporte) contra el puerto 80 del servidor web, donde espera texto plano.

Telnet no "sabe" nada de HTTP; simplemente abre un canal de texto bidireccional.

Nosotros al escribir o pegar el contenido del archivo, estamos enviando los datos de la capa de aplicación (HTTP), y la capa de aplicación del servidor la decodifica, la entiende y responde por el canal abierto.

b. ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?

- **Método:** GET.
- **Recurso:** `/http/HTTP-1.1/`

c. ¿Qué diferencias notó entre los dos casos? ¿Puede explicar por qué?

- Diferencia: En el caso de HTTP/1.0, la conexión se cerró inmediatamente después de que el servidor terminó de enviar el HTML (Connection closed).
En el caso de HTTP/1.1, la conexión permaneció abierta (viva), permitiendome pegar el comando nuevamente.
- Explicación: Esto ocurre porque la V1.1 usa conexiones persistentes (Keep-Alive). HTTP/1.1 asume por defecto que el cliente querrá pedir más cosas (imágenes, CSS, etc.) por el mismo "tubo" para no perder tiempo abriendo nuevas conexiones TCP.

d. ¿Cuál de los dos casos le parece más eficiente? Piense en el ejercicio donde analizó la cantidad de requerimientos necesarios para obtener una página con estilos, javascripts e imágenes. El caso elegido, ¿puede traer asociado algún problema?

El más eficiente es HTTP/1.1. Es mucho más rápido porque evita el costo de tiempo del Handshake de TCP para cada pequeño recurso de la página.

Posibles problemas:

1. Head-of-Line (HOL) Blocking:

HTTP/1.1 exige estrictamente que las solicitudes en una única conexión persistente se procesen en orden. Si bien la canalización permite enviar múltiples solicitudes sin esperar respuestas individuales, el servidor debe devolver las respuestas en el orden exacto solicitado. Si la primera solicitud tarda mucho, las respuestas posteriores se bloquean.

Si pedís un archivo muy pesado primero, los demás pedidos que vienen atrás en la misma conexión deben esperar a que ese termine para empezar a bajar.

2. Consumo de recursos en el servidor: Mantener miles de conexiones "abiertas y en espera" consume memoria RAM y sockets en el servidor, lo que podría saturarlo al pedo si hay demasiados clientes inactivos.

17. En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET. Para ello, será necesario utilizar la VM y la herramienta Wireshark.

Antes de iniciar considere:

- Capture los paquetes utilizando la interfaz con IP 172.28.0.1. (Menú "Capture ->Options". Luego seleccione la interfaz correspondiente y presione Start).
- Para que el analizador de red sólo nos muestre los mensajes del protocolo http introduciremos la cadena 'http' (sin las comillas) en la ventana de especificación de filtros de visualización (display-filter). Si no hiciéramos esto veríamos todo el tráfico que es capaz de capturar nuestra placa de red. De los paquetes que son capturados, aquel que esté seleccionado será mostrado en forma detallada en la sección que está justo debajo. Como sólo estamos interesados en http ocultaremos toda la información que no es relevante para esta práctica (Información de trama, Ethernet, IP y TCP). Desplegar la información correspondiente al protocolo HTTP bajo la leyenda "Hypertext Transfer Protocol".
- Para borrar la cache del navegador, deberá ir al menú "Herramientas->Borrar historial reciente". Alternativamente puede utilizar Ctrl+F5 en el navegador para forzar la petición HTTP evitando el uso de caché del navegador.
- En caso de querer ver de forma simplificada el contenido de una comunicación http, utilice el botón derecho sobre un paquete HTTP perteneciente al flujo capturado y seleccione la opción Follow TCP Stream.

a. Abra un navegador e ingrese a la URL: www.redes.unlp.edu.ar e ingrese al link en la sección "Capa de Aplicación" llamado "Métodos HTTP". En la página mostrada se visualizan dos nuevos links llamados: Método GET y Método POST. Ambos muestran un formulario.

b. Analice el código HTML

Ambos tienen un formulario.

- Formulario GET: `<form method="GET" action="...">`
- Formulario POST: `<form method="POST" action="...">`

c. Utilizando el analizador de paquetes Wireshark capture los paquetes enviados y recibidos al presionar el botón Enviar.

GET

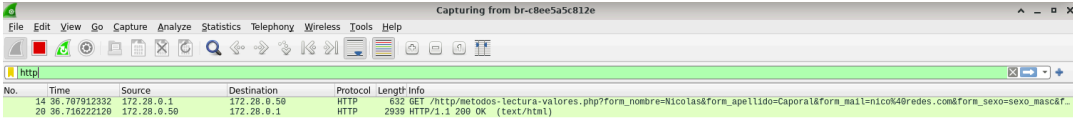


Métodos HTTP: Lectura de valores desde REQUEST

Esta página es el resultado de leer los valores recibidos en un mensaje HTTP. Desde el punto de vista del programador, puede obligarse la lectura según el método sea *GET*, *POST* u ambos. En este caso, utilizamos el último caso, es decir que este mismo código sirve para leer valores enviados utilizando GET o POST indistintamente.

Los valores recibidos son

Nombre: Nicolas
Apellido: Caporal
Email: nico@redes.com
Sexo: Masculino
Contraseña: contraseña12345
Recibir confirmaciones por email: Si



No.	Time	Source	Destination	Protocol	Length	Info
14	36.707912332	172.28.0.1	172.28.0.50	HTTP	632	GET /http/metodos-lectura-valores.php?form_nombre=Nicolas&form_apellido=Caporal&form_mail=nico@redes.com&form_sexo=sexo_masc&form_contraseña=contraseña12345&form_confirmar_mail=on
20	36.716222120	172.28.0.50	172.28.0.1	HTTP	2939	HTTP/1.1 200 OK (text/html)

POST

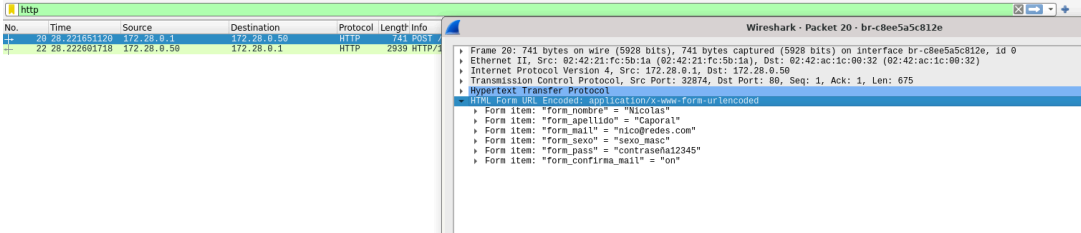


Métodos HTTP: Lectura de valores desde REQUEST

Esta página es el resultado de leer los valores recibidos en un mensaje HTTP. Desde el punto de vista del programador, puede obligarse la lectura según el método sea *GET*, *POST* u ambos. En este caso, utilizamos el último caso, es decir que este mismo código sirve para leer valores enviados utilizando GET o POST indistintamente.

Los valores recibidos son

Nombre: Nicolas
Apellido: Caporal
Email: nico@redes.com
Sexo: Masculino
Contraseña: contraseña12345
Recibir confirmaciones por email: Si



No.	Time	Source	Destination	Protocol	Length	Info
20	36.716222120	172.28.0.1	172.28.0.50	HTTP	741	POST /http/metodos-lectura-valores.php
22	36.722003170	172.28.0.50	172.28.0.1	HTTP	2939	HTTP/1.1 200 OK (text/html)

Wireshark · Packet 20 · br-c8ee5a5c812e

- Frame 20: 741 bytes on wire (5928 bits), 741 bytes captured (5928 bits) on interface br-c8ee5a5c812e, id 0
- Ethernet II, Src: 02:42:21:fc:5b:1a (02:42:21:fc:5b:1a), Dst: 02:42:ac:1c:00:32 (02:42:ac:1c:00:32)
- Internet Protocol Version 4, Src: 172.28.0.1, Dst: 172.28.0.50
- Transmission Control Protocol, Src Port: 32874, Dst Port: 80, Seq: 1, Ack: 1, Len: 675
- Hypertext Transfer Protocol
- HTTP/1.1 200 OK (text/html)
- Content-Type: text/html
- Content-Length: 2939
- Form item: "form_nombre" = "Nicolas"
- Form item: "form_apellido" = "Caporal"
- Form item: "form_mail" = "nico@redes.com"
- Form item: "form_sexo" = "sexo_masc"
- Form item: "form_contraseña" = "contraseña12345"
- Form item: "form_confirmar_mail" = "on"

d. ¿Qué diferencias detectó en los mensajes enviados por el cliente?

- **Ubicación de los datos:**
En el mensaje GET, los datos viajan dentro de la Línea de Requerimiento (Request-Line), pegados a la URL después del signo ?. En cambio, en el mensaje POST, los datos viajan en el Cuerpo del Mensaje (Entity Body), separados de los encabezados por una línea en blanco.
- **Encabezados adicionales:**
El mensaje POST incluye cabeceras obligatorias que el GET no necesita, principalmente Content-Type y Content-Length, que indica el tamaño en bytes de los datos que van en el cuerpo.
- **Visibilidad en el paquete:**
Se puede ver en ambos casos porque es HTTP, sin la S.
En Wireshark, el GET muestra toda la información en el campo "Info" de la lista de paquetes. Para el POST, tuve que desplegar la sección específica de "HTML Form URL Encoded" al final del análisis del paquete para ver el contenido.

e. ¿Observó alguna diferencia en el browser si se utiliza un mensaje u otro?

- Con el método GET, la barra de direcciones se "ensucia" mostrando todos los pares de campo=valor. Con el método POST, la URL permanece "limpia", mostrando solo la ruta del archivo .php receptor.
- Los datos enviados por **GET** quedan registrados en el historial del navegador (incluyendo contraseñas si el desarrollador cometió el error de usarlos ahí), mientras que los datos de un **POST** no son visibles en el historial de navegación

18. Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP

Set-Cookie (Del Servidor al Cliente):

Cuando el servidor quiere que el navegador "recuerde" algo, envía esta cabecera en la respuesta. Contiene un nombre, el valor seteado y opcionalmente atributos como la fecha de expiración, el dominio o si debe ser solo por HTTPS.

Cookie (Del Cliente al Servidor):

Una vez que el navegador tiene la cookie guardada, la incluirá automáticamente en el encabezado de todos los requerimientos futuros que envíe a ese mismo servidor.

Relación con HTTP:

HTTP es un protocolo "sin estado" (stateless).

Esto significa que:

- El servidor trata cada pedido como algo totalmente nuevo y aislado.
- El servidor no "sabe" por sí solo que el GET que acabás de hacer viene de la misma persona que hizo un POST hace dos segundos.

Las cookies son la solución a esta amnesia del protocolo. Permiten "atar" diferentes pedidos entre sí para crear una sesión.

19. ¿Cuál es la diferencia entre un protocolo binario y uno basado en texto? ¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?

Protocolos basados en Texto:

- Están diseñados para ser leídos por humanos.
Los comandos y datos se envían utilizando caracteres imprimibles (como ASCII o UTF-8).
- Depuración: Es muy fácil encontrar errores porque los mensajes son palabras literales (ej: GET, 200 OK, Host).
- Eficiencia: Son menos eficientes para las computadoras. El servidor tiene que parsear cadenas de texto buscando espacios o saltos de línea para entender dónde termina un encabezado y empieza otro.

Protocolos Binarios:

- Están diseñados para ser leídos por máquinas. La información se organiza en campos de bits y bytes con posiciones fijas.
Si intentás ver un mensaje binario en una terminal, verías símbolos extraños ("basura"). Necesitás herramientas como Wireshark para interpretarlos
- Son mucho más rápidos y compactos. No hay ambigüedad; el protocolo dice exactamente "los bits del 0 al 4 son el tipo de mensaje", por lo que el procesamiento es casi instantáneo para el CPU.
Al tener estructuras rígidas, son menos propensos a errores de interpretación por espacios de más o caracteres mal codificados.

De qué tipo es cada versión:

- **HTTP/1.0** : Basado en texto
- **HTTP/1.1** : Basado en texto
- **HTTP/2** : Binario

20. Responder las siguientes preguntas:

a. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0? ¿Qué sucede en HTTP/2?

(Ayuda: <https://undertow.io/blog/2015/04/27/An-in-depth-overview-of-HTTP2.html> para HTTP/2)

- **¿Existía en HTTP/1.0?:**
No originalmente. Al no ser obligatoria ni estándar, los servidores de esa época solían requerir una dirección IP única para cada sitio web.
- **Función en HTTP/1.1:**
Es obligatoria. Permite el Virtual Hosting, indicando al servidor qué dominio específico querés visitar. Así, una misma dirección IP puede alojar múltiples sitios web distintos.
- **¿Qué sucede en HTTP/2?:**
Se reemplaza técnicamente por un "pseudo-encabezado" llamado :authority. Cumple el mismo propósito, pero está adaptado a la nueva estructura binaria del protocolo.

b. En HTTP/1.1, ¿es correcto el siguiente requerimiento?

GET /index.php HTTP/1.1 User-Agent: curl/7.54.0

No, es incorrecto.

En el protocolo HTTP/1.1, la cabecera Host es estrictamente obligatoria. Si un servidor recibe esta petición, debería rechazarla devolviendo un código de error 400 Bad Request.

A diferencia de HTTP/1.0, donde el Host era opcional, en la versión 1.1 se volvió indispensable para permitir que una misma dirección IP pueda alojar múltiples dominios distintos (Virtual Hosting). Sin esa cabecera, el servidor no sabría a cuál de sus sitios web querés acceder

c. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https?

GET /index.php HTTP/1.1
Host: www.info.unlp.edu.ar

Quedaría:

:method: GET
:path: /index.php
:scheme: https
:authority: www.info.unlp.edu.ar